Unidad 2 a — Introducción a Spring Boot

1. ¿Qué es Spring Boot?

Spring Boot es una capa sobre **Spring Framework** que te permite crear aplicaciones de producción **rápido**, con:

- **Autoconfiguración**: detecta dependencias y configura beans por ti.
- **Starters**: "paquetes de dependencias" listos (`spring-boot-starter-web`, `data-jpa`, etc.).
- **Servidor embebido** (Tomcat/Jetty/Undertow): ejecutas con `java -jar`.
- **Convención sobre configuración**: valores por defecto sensatos; personalizas en `application.yml`, o `application.properties`.

Relación con Spring

- Spring Boot **no sustituye** a Spring; lo **facilita**:
  - Sigue habiendo **IoC/DI** (Inversión de Control e **inyección de dependencias**).
  - Siguen existiendo **beans**, contexto de aplicación, perfiles, etc.

---

2. Capacidades y usos principales

- **APIs REST / Web** (starter: `spring-boot-starter-web`)
- **Acceso a datos JPA** (starter: `spring-boot-starter-data-jpa`)
- **Validación** (starter: `spring-boot-starter-validation`)
- **Seguridad** (starter: `spring-boot-starter-security`)
- **Documentación OpenAPI** (springdoc)
- **Tests** (starter: `spring-boot-starter-test`)
- **Actuator / Monitorización**, **mensajería**, **batch**, **cloud**...

---

3. Estructura típica de un proyecto

```
src/
  main/java/com/ejemplo/app/
    Application.java            # @SpringBootApplication
    web/... o /controllers      # Controladores REST
    service/...                 # Servicios (lógica de negocio)
    repo/... o /repositories    # Repositorios JPA
    domain/... o /entities      # Entidades JPA
    dto/...                     # DTOs de entrada/salida
    security/...                # (opcional) Seguridad
    config/...                  # (opcional) Configuraciones
  main/resources/
    application.yml o           # Config común
    application.properties      # Config común
    application-dev.yml         # Perfil dev
    application-dev.propertie   # Perfil dev
    application-prod.yml        # Perfil prod
    application-prod.properties # Perfil prod
    db/migration/...            # (opcional) Flyway o Liquibase
  test/java/...                 # Tests
pom.xml
```

Aquí vemos: separación por capas (Controller → Service → Repository), configuración por perfiles y posibilidad de migraciones con Flyway.

---

4. Starters y autoconfiguración (qué usamos y por qué)

En `pom.xml` añades starters. Spring Boot detecta:

- Si tienes `spring-boot-starter-web` → **expone** Tomcat embebido, mapea controladores, configura Jackson (JSON).
- Si añades `data-jpa` + un driver (H2/Postgres) → **configura** EntityManager, DataSource y repositorios.

Ventaja: escribes **menos boilerplate** y te centras en la **lógica**.

---

5. Configuración por perfiles

- `application.yml`: común.
- `application-dev.yml`: dev (H2, `ddl-auto: update/create-drop`).
- `application-prod.yml`: prod (Postgres, Flyway, logs).

Activas con `SPRING_PROFILES_ACTIVE=dev|prod`.

---

6. Ejemplo guiado paso a paso — Mini-API "Tasks"

> Objetivo: construir una **API REST simple** con **Task{id, title, done}**, usando web + JPA + validación + Swagger. Duración estimada en clase: 60–90 min (perfecto para "primera API").

6.0. Crear proyecto (Initializr o manual)

Opción A — Spring Initializr

- Dependencias: **Web**, **Validation**, **JPA**, **H2**, **Lombok** (opcional), **Springdoc OpenAPI**.
- Java 25.

Opción B — pom.xml mínimo

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
```

```
  </dependency>
  <dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>3.0.3</version>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>
</dependencies>
```

¿Qué pasa aquí?

- web → controlador REST y JSON listo.
- `validation` → `@Valid`, `@NotBlank`, etc.
- `data-jpa` + h2 → persistencia en memoria para dev.
- `springdoc` → **Swagger UI** automático en `/swagger-ui.html`.

Decisión del proyecto conductor: Battleship conserva un contrato OpenAPI 3.1 estático como fuente de verdad. Springdoc aporta el visor Swagger UI, pero su generación automática está deshabilitada para evitar dos contratos.

6.1. Configuración (`application.yml` + perfil dev)

```yaml
# src/main/resources/application.yml
spring:
  application.name: tasks
server:
  port: 8080
springdoc:
  swagger-ui:
    path: /swagger-ui.html
```

```yaml
# src/main/resources/application-dev.yml
spring:
  datasource:
    url: jdbc:h2:mem:tasks;DB_CLOSE_DELAY=-1
    username: sa
    password: sa
  jpa:
    hibernate:
      ddl-auto: update
    properties:
      hibernate:
        format_sql: true
logging:
  level:
    root: INFO
```

Aquí usamos: H2 en memoria para desarrollo y **Swagger UI**.

Arranca con:

```
SPRING_PROFILES_ACTIVE=dev ./mvnw spring-boot:run
```

6.2. Entidad JPA (Task)

Entidad JPA simple con clase java yLombok

```java
// src/main/java/com/example/tasks/domain/Task.java
package com.example.tasks.domain;

import jakarta.persistence.*;
import lombok.*;

@Entity @Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Task {
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;

  @Column(nullable = false)
  private String title;

  private boolean done;
}
```

Aquí vemos: mapeo JPA simple con `@Entity`, `@Id`, y **Lombok** para reducir boilerplate.

6.3. Repositorio JPA

```java
// src/main/java/com/example/tasks/repo/TaskRepository.java
package com.example.tasks.repo;

import com.example.tasks.domain.Task;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

public interface TaskRepository extends JpaRepository<Task, Long> {
  List<Task> findByDone(boolean done);
}
```

Aquí usamos: `JpaRepository` genera CRUD básico; añadimos **consulta derivada** `findByDone`.

6.4. DTOs y validación

```java
// src/main/java/com/example/tasks/dto/CreateTaskDTO.java
package com.example.tasks.dto;

import jakarta.validation.constraints.NotBlank;

public record CreateTaskDTO(@NotBlank String title) {}
```

Aquí vemos: `@NotBlank` valida el campo; si falla, Spring lanzará `MethodArgumentNotValidException`.

6.5. Servicio (lógica de negocio)

```java
// src/main/java/com/example/tasks/service/TaskService.java
package com.example.tasks.service;

import com.example.tasks.domain.Task;
import com.example.tasks.dto.CreateTaskDTO;
import com.example.tasks.repo.TaskRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service @Transactional
public class TaskService {
  private final TaskRepository repo;
  public TaskService(TaskRepository repo) { this.repo = repo; }

  public List<Task> list(Boolean done) {
    return (done == null) ? repo.findAll() : repo.findByDone(done);
  }

  public Task create(CreateTaskDTO dto) {
    Task t = Task.builder().title(dto.title()).done(false).build();
    return repo.save(t);
  }

  public Task toggle(Long id) {
    Task t = repo.findById(id).orElseThrow(() -> new IllegalArgumentException("Task not found
    t.setDone(!t.isDone());
    return t;
  }

  public void delete(Long id) {
    if (!repo.existsById(id)) throw new IllegalArgumentException("Task not found");
    repo.deleteById(id);
  }
}
```

Aquí vemos:

- **Transaccionalidad** con `@Transactional`.
- **Reglas de negocio** simples: crear con `done=false`, `toggle` invierte estado, errores si no existe.

6.6. Controlador REST

```java
// src/main/java/com/example/tasks/web/TaskController.java
package com.example.tasks.web;

import com.example.tasks.domain.Task;
import com.example.tasks.dto.CreateTaskDTO;
import com.example.tasks.service.TaskService;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
```

```java
import java.net.URI;
import java.util.List;

@RestController
@RequestMapping("/tasks")
public class TaskController {
  private final TaskService service;
  public TaskController(TaskService service) { this.service = service; }

  @GetMapping
  public List<Task> list(@RequestParam(required = false) Boolean done) {
    return service.list(done);
  }

  @PostMapping
  public ResponseEntity<Task> create(@RequestBody @Valid CreateTaskDTO dto) {
    Task saved = service.create(dto);
    return ResponseEntity.created(URI.create("/tasks/" + saved.getId())).body(saved);
  }

  @PatchMapping("/{id}/toggle")
  public Task toggle(@PathVariable Long id) {
    return service.toggle(id);
  }

  @DeleteMapping("/{id}")
  public ResponseEntity<Void> delete(@PathVariable Long id) {
    service.delete(id);
    return ResponseEntity.noContent().build();
  }
}
```

Aquí usamos y vemos:

- `@RestController` y **rutas** REST.
- `@RequestParam` para filtrar (`/tasks?done=true`).
- `@Valid` para validar entrada.
- **Códigos HTTP correctos**: `201 Created` con `Location`, `204 No Content` en delete.

6.7. Gestión uniforme de errores (`@ControllerAdvice`)

```java
// src/main/java/com/example/tasks/web/GlobalExceptionHandler.java
package com.example.tasks.web;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.*;
import java.time.Instant;
import java.util.Map;

@ControllerAdvice
public class GlobalExceptionHandler {

  @ExceptionHandler(MethodArgumentNotValidException.class)
  public ResponseEntity<?> validation(MethodArgumentNotValidException ex) {
    String msg = ex.getBindingResult().getAllErrors().get(0).getDefaultMessage();
```

```
    return ResponseEntity.badRequest().body(Map.of(
      "error", "VALIDATION_ERROR",
      "message", msg,
      "timestamp", Instant.now().toString()
    ));
  }

  @ExceptionHandler(IllegalArgumentException.class)
  public ResponseEntity<?> badRequest(IllegalArgumentException ex) {
    return ResponseEntity.badRequest().body(Map.of(
      "error", "BAD_REQUEST",
      "message", ex.getMessage(),
      "timestamp", Instant.now().toString()
    ));
  }
}
```

Aquí vemos: un **payload de error consistente** para validación y argumentos no válidos.

6.8. Probar la API (curl / Insomnia / Swagger)

- Levanta en dev:

```
SPRING_PROFILES_ACTIVE=dev ./mvnw spring-boot:run
```

- Swagger UI: `http://localhost:8080/swagger-ui.html`
- Pruebas rápidas:

```
# Crear
curl -s -X POST http://localhost:8080/tasks \
  -H "Content-Type: application/json" \
  -d '{ "title": "Aprender Spring Boot" }' | jq

# Listar
curl -s http://localhost:8080/tasks | jq

# Toggle
curl -s -X PATCH http://localhost:8080/tasks/1/toggle | jq

# Borrar
curl -i -X DELETE http://localhost:8080/tasks/1
```

Qué observar: códigos 201/200/204, errores 400 con JSON “bonito” si envías datos inválidos.

6.9. Variantes y extensiones

- **H2 → Postgres** en `application-prod.yml` + Docker Compose.
- Añadir **seguridad** (más adelante): `spring-boot-starter-security` + JWT/Bearer.
- **DTO de salida** para no exponer entidad completa.
- **Tests**: controlador/servicio con `SpringBootTest` o `WebMvcTest`.

---

7. Fallos típicos y cómo resolverlos

- **No arranca y se cierra**: suele faltar el **driver** o la URL de BD es incorrecta.

- **`400 Bad Request` en POST**: falta un campo requerido → revisa `@Valid` y mensajes.
- **Swagger no carga**: la app no está levantada o la ruta no es `/swagger-ui.html`.
- **CORS** (si hay frontend aparte): añade configuración de CORS en un `SecurityConfig`.

---

8. Ideas clave para llevarte

- Spring Boot es **Spring con turbo**: autoconfiguración y starters.
- Construyes APIs REST con **pocas líneas**: controlador + servicio + repositorio.
- **Perfiles** te permiten cambiar configuración sin tocar código.
- **Validación** y **errores uniformes** mejoran muchísimo la calidad de la API.

Mini-check (Comprueba tu aprendizaje)

1. ¿Qué hace un **starter** de Spring Boot?
2. ¿Qué ventaja tiene `ResponseEntity.created(...)` frente a devolver 200 en un POST?
3. ¿Cómo activar un perfil distinto sin cambiar código?
4. ¿Por qué conviene usar **DTOs** en la salida?

Apéndice I: Clases vs Records: cuándo usar cada una (Entidades y DTOs)

En Java, desde la versión 16, los **records** permiten definir objetos inmutables de forma compacta. En Spring Boot se pueden usar, pero **hay que distinguir entre entidades y DTOs.**

Entidades JPA: mejor con `class`

Las **entidades** representan objetos persistentes que Hibernate debe construir, modificar y guardar. Por tanto, **necesitan un constructor sin argumentos y setters públicos o protegidos**.

```
@Entity
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Task {
  @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;
  private String title;
  private boolean done;
}
```

Por qué usar `class`:

- Hibernate necesita un **constructor vacío**.
- Puede modificar campos internamente (al hacer `UPDATE` o `MERGE`).
- Permite usar **Lombok** para reducir boilerplate.
- Facilita la **serialización/deserialización** sin errores.

```
// [X] Esto daría problemas en Hibernate:
@Entity
public record Task(Long id, String title, boolean done) {}
```

Problema: Hibernate no puede crear la entidad porque los campos son `final` y no hay constructor sin argumentos. El error típico sería:

```
org.hibernate.InstantiationException: No default constructor for entity: Task
```

En resumen: usa `class` para tus entidades JPA.

DTOs: perfectos para `record`

Los **DTOs (Data Transfer Objects)** solo transportan datos entre capas (por ejemplo, del servicio al controlador). No se modifican ni se guardan en la base de datos, por lo que **ser inmutables es una ventaja**.

```
// DTO de entrada
public record CreateTaskDTO(@NotBlank String title) {}

// DTO de salida
public record TaskDTO(Long id, String title, boolean done) {}
```

Uso en el controlador:

```
@PostMapping
public ResponseEntity<TaskDTO> create(@Valid @RequestBody CreateTaskDTO dto) {
    Task saved = service.create(dto);
    TaskDTO out = new TaskDTO(saved.getId(), saved.getTitle(), saved.isDone());
    return ResponseEntity.created(URI.create("/tasks/" + saved.getId())).body(out);
}
```

[OK] **Ventajas de usar `record` en DTOs:**

- Código mucho más limpio (solo declaras los campos).
- Inmutables → más seguros frente a modificaciones accidentales.
- Perfectos para representar **datos de entrada/salida** o **respuestas REST**.
- Generan automáticamente `equals`, `hashCode`, `toString`.

Resumen comparativo

Uso	Tipo	Mutabilidad	Constructor vacío	Recomendado para
Entidades	`class`	Mutable	Sí	Persistencia con JPA
DTOs	`record`	Inmutable	No necesario	Transporte de datos, respuestas REST
Proyecciones de lectura	`record`	Inmutable	No necesario	Consultas `@Query`, vistas de solo lectura

---

Conclusión didáctica

- **Entidades → `class` + Lombok.** “Hibernate necesita construir y actualizar.”
- **DTOs → `record`.** “Compacto, inmutable y perfecto para devolver datos.”
